

Синхронизация горутин

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL



Матвей Ефимов

О спикере:

- Преподаватель с более чем 5-летним опытом в IT-образовании
- Разработчик учебных программ и эксперт в Go, Python, SQL
- Фриланс-программист и наставник студентов от школ до вузов

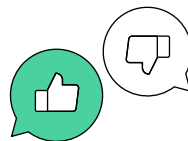


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры **Google Chrome** или **Microsoft Edge**
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение **МТС Линк** на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по [ссылке](#)

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара – 1,5 часа
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в личном кабинете





**Какие у вас ожидания
от вебинара?**



Ваши вопросы?

Цели занятия

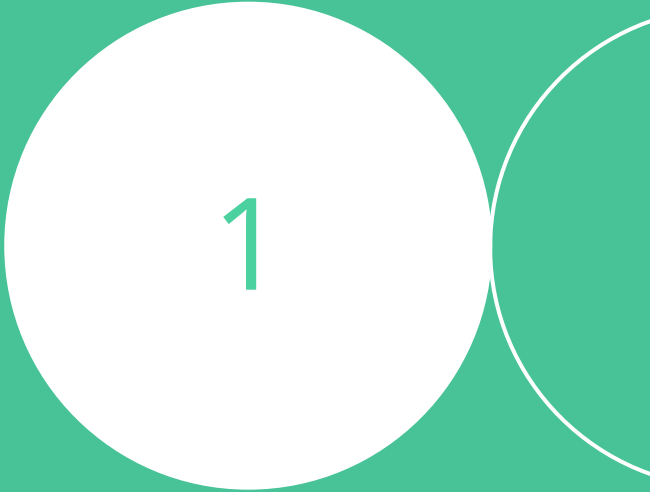
- Понять, в чём заключается проблема конкурентного доступа к переменным при выполнении горутин
- Разобраться, как возникают гонки данных и к каким ошибкам они приводят
- Освоить базовые средства синхронизации в Go: `sync.WaitGroup` и `sync.Mutex`, оценить возможности и ограничения каждого
- Научиться применять `WaitGroup` и `Mutex` на практике для безопасной работы с общими ресурсами
- Проанализировать примеры конкурентного кода и выбрать подходящий механизм синхронизации для каждой ситуации.

План занятия

- 1 Проблема конкурентного доступа
- 2 Взаимные блокировки (Deadlocks)
- 3 Утечки горутин (Goroutine Leaks)
- 4 Голодание (Starvation)
- 5 Способ решения 1: `sync.WaitGroup`
- 6 Способ решения 2: `sync.Mutex`
- 7 Способ решения 3: `atomic`
- 8 Способ решения 4: каналы
- 9 Практический разбор
- 10 Итоги



Проблема конкурентного доступа



1

Что такое синхронизация горутин

Синхронизация горутин — управление порядком и безопасностью выполнения горутин, если они работают с одними и теми же данными или зависят друг от друга

Синхронизация нужна, чтобы горутины:

- не мешали друг другу при доступе к общим переменным
- выполнялись в правильной последовательности, если одна горутина зависит от результата другой
- завершились корректно, прежде чем программа продолжит выполнение

Почему без синхронизации не обойтись

Когда несколько горутин работают с одними и теми же данными, важно контролировать:

- что происходит, если они обращаются к ресурсу одновременно
- что делать, если одна горутина завершилась, а другая продолжает работу

Гонки данных (Data Races)

Гонка данных — одна из самых распространённых проблем при работе с горутинами

- Это ситуация, когда две или более горютины одновременно обращаются к одной и той же переменной — и хотя бы одна из них записывает значение. Всё это происходит без синхронизации

Почему это опасно

- Порядок операций становится непредсказуемым
- Нельзя быть уверенным в содержимом переменной
- Ошибки проявляются случайно и нестабильно
- Такие ошибки особенно сложны для отладки

Пример гонки данных

- Создадим переменную-счётчик, к которой будут обращаться несколько горутин

Что делает код

- Объявляется переменная counter, которую нужно увеличить 5 раз — по одному разу в каждой горутине
- Запускается 5 горутин в цикле
- После этого программа ждёт 1 секунду, чтобы все горутин завершили работу, и выводит значение счётчика

```
package main

import (
    "fmt"
    "time"
)

func main() {
    counter := 0

    for i := 0; i < 5; i++ {
        go func() {
            counter++
        }()
    }

    time.Sleep(1 * time.Second)
    fmt.Println("Counter:", counter)
}
```

Ответьте на вопрос

Какое значение мы ожидаем увидеть на экране?

```
package main

import (
    "fmt"
    "time"
)

func main() {
    counter := 0

    for i := 0; i < 5; i++ {
        go func() {
            counter++
        }()
    }

    time.Sleep(1 * time.Second)
    fmt.Println("Counter:", counter)
}
```

Результат выполнения

Кажется, что значение должно быть 5. Но на практике можно получить:

- 2, 3, 4, 5 и даже 1 (редко — на новых, быстрых процессорах)

Это происходит из-за гонки данных: несколько горутин одновременно обращаются к общей переменной без синхронизации

```
package main

import (
    "fmt"
    "time"
)

func main() {
    counter := 0

    for i := 0; i < 5; i++ {
        go func() {
            counter++
        }()
    }

    time.Sleep(1 * time.Second)
    fmt.Println("Counter:", counter)
}
```


Почему результат непредсказуем

Операция `counter++` не является атомарной. Она включает три действия:

- Чтение текущего значения `counter`
- Увеличение значения на 1
- Запись нового значения обратно

- Когда несколько горутин выполняют `counter++` одновременно, они читают старое значение почти одновременно. Затем каждая горутина считает новое значение и записывает одно и то же, затирая результат предыдущей
- Так возникает гонка данных

Как отловить гонку данных

Go предоставляет встроенный инструмент — детектор гонок

Чтобы запустить проверку, используйте флаг `-race`: **`go run -race main.go`**

- Вы получите предупреждение, если в программе есть гонка данных, и увидите, где именно она произошла
- Это полезный инструмент для отладки. Если работаете с параллельным кодом — запускайте тесты и файлы с флагом `-race`. Это обязательная практика

Пример: общий срез и горютины

Что происходит

- В пять горютин передаётся значение `i`
- Каждая горютина добавляет своё значение в общий срез `data`
- При запуске с флагом `-race Go` предупреждает о гонке данных

Почему возникает гонка

Операция **append** может перераспределять память, если срез превышает ёмкость

- Если несколько горютин выполняют **append** одновременно, это может:
- повредить структуру среза
- привести к краху программы
- вызвать некорректные данные

```
package main

import (
    "fmt"
    "time"
)

func main() {
    data := []int{}

    for i := 0; i < 5; i++ {
        go func(val int) {
            data = append(data, val)
        }(i)
    }

    time.Sleep(1 * time.Second)
    fmt.Println("Data:", data)
}
```



**Возможна ли гонка данных,
если переменная читается в нескольких
горутинах одновременно?**

Ответ

Если нет записи — гонки не будет

- Одновременное чтение переменной безопасно, если она не изменяется
- Гонка возникает при любой записи — даже если остальные операции только читают

Как понять, что возникла гонка данных

Признаки гонки:

- переменная не увеличивается до ожидаемого значения
- программа ведёт себя по-разному при каждом запуске
- появляются ошибки вроде `index out of range` или `invalid memory address`

→ Важно: не полагайтесь на визуальные симптомы — всегда используйте `-race`

Почему важно говорить о гонках данных с самого начала

- Гонки данных — частая причина нестабильности многопоточных программ
- Даже логически корректный код может работать непредсказуемо, если не учтено параллельное выполнение
- Ошибки сложно выявить на тестах — они проявляются под нагрузкой или в проде



Профилактика важнее отладки: понимание гонок данных с самого начала — залог стабильной работы программы

Что происходит в этом примере?

```
package main

import (
    "fmt"
)

func main() {
    x := 0

    go func() {
        x = 1
    }()

    fmt.Println(x)
}
```


Что происходит в этом примере?

- Печать может выполняться до или после записи
- Нет синхронизации между операциями
- Возможен вывод 0 или 1
- Детектор гонок выдаёт: **Found 1 data race(s)**
- Даже такой код считается гонкой данных

```
package main

import (
    "fmt"
)

func main() {
    x := 0

    go func() {
        x = 1
    }()

    fmt.Println(x)
}
```

Резюмируем

Гонка данных возникает, когда:

- переменная используется несколькими горутинами одновременно
- хотя бы одна из них записывает значение
- нет синхронизации

В таких случаях поведение программы становится недетерминированным: вы не можете предсказать результат — он зависит от порядка выполнения

Чтобы избежать проблем:

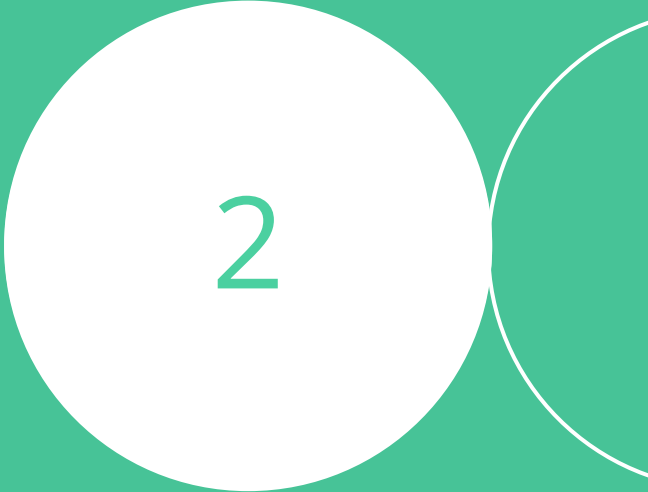
- всегда запускайте код с флагом **-race**
- помните: синхронизация — это не про ускорение, а про контроль и безопасность



Ваши вопросы?

Взаимные блокировки

(Deadlocks)



2

Что такое взаимная блокировка

Взаимная блокировка (deadlock) — это ситуация, при которой:

- несколько горутин ждут друг друга
- ни одна не может продолжить выполнение

- Программа не завершается, но и не движется — выполнение «зависает»
- Это не ошибка в привычном смысле, а следствие нарушения синхронизации

Как выглядит взаимная блокировка в жизни

→ **Представьте: два человека зашли в узкий коридор с разных сторон**

- Один не может пройти, потому что мешает другой — и наоборот
- Они вежливо ждут, кто уступит, но никто не двигается. Это может продолжаться бесконечно

→ **Так же происходит в программах:**

- две (или более) горютины удерживают ресурсы и ждут, когда другая освободит
- но никто не освобождает — выполнение «зависает»

Пример взаимной блокировки

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var mu1 sync.Mutex
    var mu2 sync.Mutex

    go func() {
        mu1.Lock()
        fmt.Println("Горутинa 1 захватила mu1")

        // имитируем небольшую задержку
        // чтобы горутинa 2 успела захватить mu2
        // и мы попали в deadlock
        defer mu1.Unlock()
```

```
mu2.Lock()
    fmt.Println("Горутинa 1 захватила mu2")
    mu2.Unlock()
}()

go func() {
    mu2.Lock()
    fmt.Println("Горутинa 2 захватила mu2")

    defer mu2.Unlock()

    mu1.Lock()
    fmt.Println("Горутинa 2 захватила mu1")
    mu1.Unlock()
}()

// Ждём завершения горутин
var wg sync.WaitGroup
wg.Add(1)
wg.Wait()
}
```

Пример взаимной блокировки



У нас есть два мьютекса: `mu1` и `mu2`

- Первая горутина захватывает `mu1`, потом пытается захватить `mu2`
- Вторая — наоборот: сначала `mu2`, потом `mu1`



Если одна горутина успевает захватить первый мьютекс, а вторая — второй, обе начинают ждать освобождения. Но этого не происходит: каждая блокирует путь для другой



Программа «зависает» — это и есть deadlock

Горутина 2 захватила `mu2`

Горутина 1 захватила `mu1`

fatal error: all goroutines are asleep – deadlock!

Почему deadlock трудно заметить

→ **Визуально программа выглядит нормально — без ошибок**, может даже завершиться «успешно»

→ **Но:**

- иногда всё работает
- иногда — зависает

→ **Это и делает проблему сложной:**

- deadlock проявляется не всегда
- это недетерминированная ошибка — и в этом её сложность

Можно ли отследить взаимную блокировку

→ Go может сам определить deadlock и выдать сообщение в stderr:

```
fatal error: all goroutines are asleep – deadlock!
```

→ **Но это работает не всегда:**

- если хотя бы одна горутина остаётся активной (например, с таймером или select), Go не определит взаимную блокировку

Как ещё можно получить взаимную блокировку

- Даже если просто неправильно спроектировать передачу данных через каналы — можно получить deadlock
- Что не так в этом фрагменте кода с каналом?

```
package main

import "fmt"

func main() {

    ch := make(chan int)

    ch <- 42

    fmt.Println("Это сообщение никогда не будет напечатано")

}
```

Почему программа зависает

- Никто не читает из канала
- Мы пытаемся записать значение в канал `ch`, но нет ни одной горутины, которая это значение примет
- Канал небуферизованный — операция `ch <- 42` будет ждать, пока кто-то не выполнит `<- ch`, но такого кода нет

→ В итоге программа зависает — происходит deadlock

```
fatal error: all goroutines are asleep – deadlock!
```

Пример, где deadlock неочевиден

- Мы создаём горутину, которая один раз записывает 1 в канал
- Затем читаем из канала — всё работает
- А потом пытаемся прочитать снова
- **Что здесь не так**
 - Никто уже ничего не пишет
 - Горутина завершилась
 - Канал пуст
 - Вторая операция чтения зависает
- Это тоже приводит к взаимной блокировке

```
package main

import "fmt"

func main() {
    ch := make(chan int)

    go func() {
        ch <- 1
    }()

    val := <-ch
    fmt.Println("Прочитали из канала:",
val)

    <-ch // Второе чтение — но никто уже
не пишет!
}
```

Частые причины deadlock'ов

Типичные шаблоны, в которых это возникает:

- Неправильный порядок захвата мьютексов. Один поток захватывает A, потом B — другой, наоборот: B, потом A
- Ожидание от горутин, которая тоже ждёт. Например, горутина читает из канала, но сама ждёт сигнал по тому же каналу
- Никто не читает из канала. Вы отправили значение — но никто не получает
- Заккрытие канала не в нужный момент. Канал закрывается до завершения всех горутин, и они не могут писать в него

Как избежать взаимных блокировок

Базовые рекомендации:

- **Соблюдайте единый порядок захвата ресурсов**

Всегда захватывайте мьютексы в одном порядке в разных участках кода

- **Используйте тайм-ауты при ожидании**

Особенно важно при работе с каналами и структурами синхронизации

- **Сокращайте область блокировки**

Чем меньше кода между Lock и Unlock, тем ниже риск deadlock

- **Анализируйте, кто кого ждёт**

Схемы и визуализация помогут выявить зависимость между горутинами

Небольшое упражнение



Представьте ситуацию:

- У вас есть две горютины
- Каждая захватывает по два ресурса
- Нужно построить порядок захвата, чтобы избежать взаимной блокировки



Подумайте: можно ли задать единый порядок



Например, всегда сначала А, потом В

Выводы

Давайте подведём итоги:

- Deadlock — ситуация, когда горутины ждут друг друга, но никто не может продолжить
- Go иногда обнаруживает deadlock, но не всегда
- Причины: пересекающиеся мьютексы, блокировка на канале, ожидание от несуществующей горутины
- Программа при этом не падает — она просто зависает
- Предотвратить проще, чем исправить



Ваши вопросы?

Утечки горутин

(Goroutine Leaks)



3

Утечки горутин (Goroutine Leaks)

- Горутина — это очень лёгкий поток. Их можно запускать сотнями, тысячами, даже миллионами. Это удобно
- Но из-за этой «лёгкости» к горутинам часто относятся нестрого: запускают — и забывают
- Ключевой момент: забыли — получили утечку горутины

Что такое утечка горутины

- **Утечка горутины** — когда горутина запускается, но никогда не завершается, хотя должна была
- Горутина зависает, остаётся жить в фоне, не принося пользы. Со временем таких горутин становится всё больше
- В результате программа потребляет больше памяти и CPU, может зависнуть или вовсе «упасть»

Когда возникают утечки горутин

Чаще всего утечки происходят, если:

- горутина заблокирована на чтении из канала, но никто не пишет в канал
- горутина ждёт сигнал, который никогда не придёт
- не передан или не проверяется контекст отмены (context)
- забыли закрыть канал или горутину после выхода из цикла

Пример: горутина зависает при чтении из канала

```
package main

import (
    "fmt"
    "time"
)

func worker(stop chan struct{}) {
    for {
        select {
        case <-stop:
            fmt.Println("Worker завершился")
            return
        case <-time.After(1 * time.Second):
            fmt.Println("Worker делает работу")
        }
    }
}
```

```
func main() {
    stop := make(chan struct{})

    go worker(stop)

    // Не останавливаем воркера — он будет жить
    // вечно!
    time.Sleep(3 * time.Second)
    fmt.Println("Завершаем main без остановки
worker")
}
```

Пример: горутина зависает при чтении из канала

- Создан канал `ch`, запущена горутина, которая ждёт значение из канала
- Но никто ничего в канал не отправляет
- Горутина остаётся активной — продолжает «жить» после завершения `main`
- В реальном сервере она бы потребляла ресурсы

→ Это и есть утечка горутины

Реалистичный пример утечки: таймер с каналом

```
package main

import (
    "fmt"
    "time"
)

func worker(stop chan struct{}) {
    for {
        select {
        case <-stop:
            fmt.Println("Worker
завершился")
            return
        case <-time.After(1 * time.Second):
            fmt.Println("Worker делает
работу")
        }
    }
}
```

```
func main() {
    stop := make(chan struct{})

    go worker(stop)

    // Не останавливаем воркера — он
    // будет жить вечно!

    time.Sleep(3 * time.Second)

    fmt.Println("Завершаем main без
остановки worker")
}
```

Реалистичный пример утечки: таймер с каналом



Что здесь не так

- Воркера запускаем, но не останавливаем
- Он продолжает работать даже после выхода из `main`
- В реальном сервере такая горутина будет жить вечно
- Это утечка: горутина работает, но уже не нужна



**Как можно было бы корректно
завершить воркера?**

Реалистичный пример утечки: таймер с каналом

Правильный ответ — послать сигнал через stop

- Вот как это можно было сделать:

```
close(stop)
```

Пример утечки: горутина в бесконечном ожидании

```
func listen(ch chan string) {  
    for msg := range ch {  
        fmt.Println("Получено:", msg)  
    }  
  
    fmt.Println("Горутина завершена")  
}
```

Что делает эта функция:

- Слушает канал ch
- Обрабатывает каждое сообщение в цикле range
- Завершается только если канал будет закрыт

Почему это утечка

- Если канал не закрыт — цикл никогда не завершится
- Даже после завершения `main` горутина будет работать
- **При масштабировании ситуация ухудшается:**
 - Вы запускаете по горутине на каждого клиента или событие
 - Канал никто не закрывает
 - В системе остаются десятки или сотни «зависших» горутин
- Это и есть утечка — лишняя, не завершившаяся работа в фоне

Пример утечки: select без условия выхода

Что происходит:

- Горутина читает из канала input
- Цикл for — бесконечный
- Условия завершения не предусмотрено

```
func doWork(input chan int) {  
    for {  
        select {  
            case val := <-input:  
                fmt.Println("Получено значение:", val)  
        }  
    }  
}
```

Пример утечки: select без условия выхода

Почему это утечка

- Канал может быть закрыт — но это никак не обрабатывается
- Мы не предусмотрели сигнал завершения (например, done)
- Горутина остаётся «живой» и продолжает ждать
- Это шаблонная ошибка — особенно при работе с select
- Чтобы избежать утечки, всегда добавляйте кейс для выхода из select

```
func doWork(input chan int) {  
    for {  
        select {  
            case val := <-input:  
                fmt.Println("Получено значение:", val)  
        }  
    }  
}
```


Как предотвратить утечки горути

- 1 Контроль завершения: используйте сигнальные каналы (**done chan struct{}**), таймеры, внешние сигналы. Горутина должна знать, когда пора завершиться
- 2 Контекст (**context.Context**): не запускайте горутины без контекста. Он позволяет передавать дедлайны и команды на отмену
- 3 Ограничивайте число горути: если запускаете горутины в цикле — ограничьте их количество. Используйте пул или буферизированный канал как семафор
- 4 Закрывайте каналы: если используете range ch, не забудьте: кто-то должен закрыть канал. Иначе горутина «зависнет» навсегда

Решение: закрытие канала

```
package main

import (
    "fmt"
    "time"
)

func worker(stop chan struct{}) {
    for {
        select {
        case <-stop:
            fmt.Println("Worker завершился")
            return
        case <-time.After(1 * time.Second):
            fmt.Println("Worker делает работу")
        }
    }
}
```

```
func main() {
    ch := make(chan int)

    go func() {
        for val := range ch {
            fmt.Println("Получено:", val)
        }
        fmt.Println("Горутина завершена")
    }()

    ch <- 1
    ch <- 2
    close(ch) // теперь горутина завершится
}
```

Решение: закрытие канала

Почему это работает

- Цикл `range ch` завершится, когда канал закроется
- Горутина выполнит `return` — и завершится корректно
- Никакой утечки не произойдёт
- Закрытие канала важно, если горутина на нём «висит»



**Что нужно предусмотреть при запуске
воркера, который только читает
из канала?**

Что нужно предусмотреть

Возможность корректно завершить чтение:

- закрытие канала (`close(ch)`)
- сигнал завершения через `context.Done()`
- использование таймера
- другой внешний сигнал для выхода из чтения

Как диагностировать утечки горутин

- Следите за количеством активных горутин. Используйте в отладке:

```
fmt.Println("Горутины:", runtime.NumGoroutine())
```

Если программа «простаивает», а число горутин растёт — это признак утечки

- Подключайте pprof. Используйте `net/http/pprof`. Профили покажут, какие горутинны живы, где заблокированы и где «висят»

Выводы

- 1 **Горутина**, которая **не завершилась** и **не приносит пользы** — это **утечка**
- 2 **Утечки** происходят **тихо, без ошибок**, но приводят к серьёзным последствиям
- 3 **Чаще всего** возникают из-за: **отсутствия сигнала завершения** и **заблокированных** или **не закрытых каналов**
- 4 **Предотвратить проще, чем искать**: продумывайте завершение при проектировании горутины



Ваши вопросы?

Голодание

(Starvation)



4

Что такое голодание (starvation)

- Вы запускаете пул воркеров или используете мьютексы для доступа к ресурсу
- Всё выглядит корректно: очередь, ожидание, работа
- Но при большой нагрузке или особенностях планировщика некоторые горютины не получают доступа к ресурсу
- **Программа не зависает, но:**
 - часть горютин бесконечно ждёт
 - ресурс освобождается, но «до них» он не доходит
 - это и называется голоданием — горютина «умирает с голоду»

Почему возникает голодание

1

Нечестные очереди

Планировщик Go не гарантирует справедливость:

та горутина, что ждёт дольше, не обязательно получит ресурс первой

2

Неравномерная работа

Быстрая горутина может постоянно захватывать и освобождать ресурс, получая преимущество

3

Приоритетные пути

Некоторые горутины могут получать доступ быстрее — остальные остаются «внизу»

4

Ошибка в логике работы с `select`, таймерами и блокировками

Если одна горутина стабильно выигрывает гонку — другие остаются без ресурса

Пример голодания: мьютекс и множество горутин

```
package main
import (
    "fmt"
    "sync"
    "time"
)
func main() {
    var mu sync.Mutex
    var wg sync.WaitGroup

    // Запускаем 10 горутин, одна из которых — "особенная"
    for i := 0; i < 9; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            for j := 0; j < 1000; j++ {
                mu.Lock()
                // короткое важное действие
                time.Sleep(time.Millisecond)
                mu.Unlock()
                time.Sleep(time.Millisecond)
            }
            fmt.Println("Горутина", id, "завершилась")
        }(i)
    }
}
```

```
wg.Add(1)
go func() { // вот "особенная" горутина
    defer wg.Done()
    mu.Lock()
    fmt.Println("Особенная горутина работает")
    mu.Unlock()
    fmt.Println("Особенная горутина завершилась")
}()

wg.Wait()
}
```

Пример голодания: мьютекс и множество горутин

Условие:

- Есть общий мьютекс
- Запущено множество горутин, каждая захватывает и отпускает мьютекс в цикле
- Одна «особенная» горутина подключается позже

Что происходит:

- Все горутины продолжают быстро захватывать и отпускать мьютекс
- «Особенная» горутина не успевает войти в критическую секцию
- Она ждёт бесконечно — ресурс освобождается, но перехватывается другими

Это и есть голодание: горутина жива, но не может выполнить свою работу

Пример голодания: конкуренция за канал

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ch := make(chan int, 1)

    go func() {
        for {
            ch <- 1
            time.Sleep(time.Millisecond)
        }
    }()
}
```

```
// несколько горутин чтения
for i := 0; i < 100; i++ {
    go func(id int) {
        for {
            select {
            case <-ch:
                fmt.Println("Горутина", id,
                    "прочитала")
            default:
                // не дождалась, ушла в другую
                // работу
                time.Sleep(time.Millisecond)
            }
        }
    }(i)
}

time.Sleep(time.Duration(0.1 *
    float64(time.Second)))
fmt.Println("Завершаем main")
}
```

Пример голодания: конкуренция за канал

Что происходит:

- Одна горутина пишет в канал
- 100 горутин пытаются читать
- **select { case <-ch: default: }** приводит к тому, что ресурс (канал) достаётся только тем, кто быстрее
- Остальные не читают и выпадают в **default-путь**, никогда не получая данные

Результат — голодание части горутин

Почему важно учитывать голодание

- 1 **Пул засоряется** — воркер не получает задачу и простаивает бесконечно
- 2 **Логика нарушается** — вы ожидаете, что все горютины обработают задачи, но часть из них остаётся без работы
- 3 **Баги трудно выявить** — проявляются только под нагрузкой или в продакшене, что затрудняет диагностику

Как диагностировать голодание горутин

1 Логирование

Записывайте активность горутин, чтобы отследить, кто выполняется, а кто простаивает

2 `runtime.NumGoroutine()`

Резкий рост числа горутин может указывать на зависшие или голодающие процессы

3 Профайлер `pprof`

Помогает выявить «горячие» и «молчаливые» горутин — те, кто активно работает и те, кто бесконечно ждёт ресурс

Как предотвратить голодание горутин

1

Справедливая очередь (fair queue)

Формируйте задачи в порядке поступления, чтобы каждая горутина получила доступ к ресурсу

2

Паузы и backoff

Если горутина не получила ресурс — делайте паузу, чтобы дать шанс другим

3

Ограничение конкуренции

Используйте семафоры или буферизированные каналы, чтобы сократить количество одновременно ожидающих

4

Избегайте блокирующих циклов без default

Это снижает вероятность того, что одни и те же горутинны будут постоянно выигрывать гонку

5

Периодическая компенсация

Отслеживайте, кто давно не выполнялся, и давайте им приоритет

Кейс: как реализовать Paused Backoff

Представьте пул из N воркеров: каждому поступает задача — он ждёт Lock

- Но один из воркеров работает медленно и часто не успевает захватить Lock
- Быстрые горутины «перехватывают» ресурс снова и снова

Вопрос: Как можно реализовать Paused Backoff, чтобы дать медленному воркеру шанс?

- Цель — понять, как замедлить быстрых и выровнять доступ к ресурсу

Резюмируем тему голодания

- Голодание — ситуация, когда горутина не получает доступ к ресурсу из-за несправедливого распределения
- Программа продолжает работать, но часть горутин «умирает с голоду»
- Основные причины:
 - жёсткая конкуренция
 - неравномерность выполнения
 - неэффективные конструкции синхронизации

Резюмируем тему голодания



Симптомы:

- незавершённые процессы
- необработанные задачи из очереди



Что помогает:

- таймауты и context
- справедливая очередь
- честный семафор
- backoff и приоритет по времени ожидания



Ваши вопросы?

Перерыв

10-15 минут



Способ Решения 1

`sync.WaitGroup`



5

Принцип работы sync.WaitGroup

- 1 Вы запускаете несколько горутин и хотите дождаться, пока они завершатся
- 2 Вы не знаете, сколько времени займёт каждая, но знаете их количество
- 3 Что делает WaitGroup:
 - Это счётчик активных горутин
 - Перед запуском — увеличиваем счётчик методом Add(n)
 - Каждая горутина по завершении вызывает Done() — счётчик уменьшается
 - Основная горутина вызывает Wait() и блокируется, пока счётчик не станет равен нулю

Результат: синхронизация завершения нескольких горутин без лишних ожиданий

Синтаксис использования sync.WaitGroup

Три ключевых метода:

- Add(n) — увеличивает счётчик на n, вызывается до запуска горутин
- Done() — уменьшает счётчик на 1, вызывается в конце работы горутин
- Wait() — блокирует выполнение, пока счётчик не обнулится

```
var wg sync.WaitGroup
wg.Add(n)    // увеличиваем счётчик на n
go func() {
    defer wg.Done() // уменьшаем счётчик, когда горутина завершится
    // работа
}()
wg.Wait()    // ждём, пока счётчик не станет ноль
```

Пример: ожидание завершения всех горутин

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup

    for i := 1; i <= 5; i++ {
        wg.Add(1) // перед запуском каждой горутины
        go func(id int) {
            defer wg.Done() // сигнал о завершении
            fmt.Printf("Горутина %d начала работу\n", id)
        }(i)
    }

    wg.Wait() // ждём всех
    fmt.Println("Все горутины завершились")
}
```

Пример: ожидание завершения всех горутин



Что происходит:

- Запускается 5 горутин
- Каждая выполняет работу и «засыпает» на 1 секунду
- После завершения вызывает Done()
- Основной поток ждёт завершения всех с помощью Wait()

Почему важно использовать `defer wg.Done()`

- Если не вызвать `Done()`, метод `Wait()` никогда не завершится
- Это приведёт к «зависанию» основного потока и невозможности продолжить выполнение программы
- Хорошая практика — вызывать `defer wg.Done()` в начале горутины, сразу после её запуска
- Это гарантирует, что `Done()` будет вызван даже при выходе из горутины с ошибкой или `return`

Пример: параллельный сбор данных с сайтов

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func fetch(url string, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Println("Получаем данные с", url)
    time.Sleep(time.Millisecond * 500) // эмуляция
    запроса
    fmt.Println("Данные с", url, "получены")
}
```

```
func main() {
    var wg sync.WaitGroup

    urls := []string{
        "https://site1.com",
        "https://site2.com",
        "https://site3.com",
        "https://site4.com",
    }

    for _, url := range urls {
        wg.Add(1)
        go fetch(url, &wg)
    }

    wg.Wait()
    fmt.Println("Все данные получены")
}
```

Пример: параллельный сбор данных с сайтов

Что важно:

- Каждый сайт обрабатывается в отдельной горутине
- WaitGroup передаётся по указателю, чтобы все горутинны работали с одной и той же структурой
- Если передать по значению — каждая горутинна получит копию, Wait() не сработает корректно

Пример: ожидание горутин с аргументами

```
for i := 0; i < 5; i++ {  
    wg.Add(1)  
    go func() {  
        defer wg.Done()  
        fmt.Println(i) // опасно!  
    }()  
}
```

Что может произойти:

5
5
5
5
5

Пример: ожидание горутин с аргументами

Почему так:

- Переменная `i` захватывается по ссылке
- К моменту выполнения горутин `i` уже изменилась

Как исправить:

```
for i := 0; i < 5; i++ {  
    wg.Add(1)  
    go func(id int) {  
        defer wg.Done()  
        fmt.Println(id)  
    }(i)  
}
```



Теперь каждая горутина получает свою копию значения `i`

Как нельзя использовать WaitGroup



Неправильный пример:

```
var wg sync.WaitGroup

go func() {
    wg.Add(1) // нельзя вызывать Add внутри горутины
    defer wg.Done()
}()
```

Почему это ошибка:

- Метод Add() вызывается после запуска горутины
- Основной поток может вызвать Wait() до того, как Add(1) выполнится
- В результате Wait() может завершиться преждевременно — Done() будет вызываться на нулевом счётчике

Правильно: всегда вызывайте wg.Add() до запуска go func

Когда применять `sync.WaitGroup`

- Когда нужно дождаться завершения всех горутин
- Когда горутин работают независимо друг от друга
- Когда не используются каналы, очереди или таймеры
- Когда важна простота и прозрачность кода

- `WaitGroup` — удобный инструмент для управления завершением множества параллельных операций
- Особенно эффективен, когда не требуется передавать данные между горутин



**У вас есть 10 горутин. Каждая делает
запрос к ресурсу. Как организовать
контроль завершения?**

Что мы узнали о WaitGroup

sync.WaitGroup — простой инструмент синхронизации, который позволяет дождаться завершения всех горутин. Он не передаёт данные и не управляет ресурсами, но помогает организовать параллельную работу

Разобрали:

- как работает WaitGroup
- как правильно запускать горутин
- типичные ошибки и способы их избежать
- примеры из практики



Ваши вопросы?

Способ Решения 2

sync.Mutex



6

Зачем нужен Mutex в Go

- **Когда мы запускаем несколько горутин, они могут обращаться к одним и тем же данным в памяти**
- **Например:**
 - общий счётчик
 - общая карта
 - другой разделяемый ресурс
- Горутины могут вмешиваться друг в друга — одна читает или пишет данные, а вторая делает это одновременно. Это приводит к гонкам данных

Что такое Mutex

- 1 **Mutex** — сокращение от *mutual exclusion* («взаимное исключение»). Позволяет только одной горутине получить доступ к критической секции кода
- 2 Мы говорим: «Сюда может зайти только один».

Синтаксис объявления Mutex

```
var mu sync.Mutex
```

Как работает мьютекс

- 1 Чтобы захватить мьютекс, вызываем метод `mu.Lock()`
- 2 Чтобы отпустить — `mu.Unlock()`
- 3 Важно:
 - Каждая блокировка должна иметь соответствующую разблокировку
 - Иначе возможна взаимная блокировка
- 4 Что происходит без мьютекса:
 - Представим, что несколько горутин увеличивают общий счётчик одновременно
 - Если не использовать `Mutex`, возможны ошибки гонки данных

Пример без мьютекса

```
package main

import (
    "fmt"
    "time"
)
var counter int
func increment() {
    for i := 0; i < 1000; i++ {
        counter++
    }
}
func main() {
    for i := 0; i < 10; i++ {
        go increment()
    }
    time.Sleep(2 * time.Second)
    fmt.Println("Counter:", counter)
}
```

Почему результат меньше 10 000

- В примере запускается 10 горутин
- Каждая увеличивает переменную counter на 1 000
- Ожидаем: $10 \times 1\,000 = 10\,000$
- Но итоговое значение меньше. Почему?
 - Несколько горутин одновременно читают, изменяют и записывают значение
 - Эти действия не атомарны
 - Пока одна горутина читает counter, другая может его уже изменить
- Это и есть гонка данных

Как исправить гонку данных с помощью Mutex

Оборачиваем изменение переменной в `mu.Lock()` и `mu.Unlock()`

```
package main

import (
    "fmt"
    "sync"
    "time"
)

var (
    counter int
    mu      sync.Mutex
)
```

```
func increment() {
    for i := 0; i < 1000; i++ {
        mu.Lock()
        counter++
        mu.Unlock()
    }
}

func main() {
    for i := 0; i < 10; i++ {
        go increment()
    }

    time.Sleep(2 * time.Second)
    fmt.Println("Counter:", counter)
}
```

Как избежать ошибок при работе с мьютексами

- 1 Мы обернули изменение counter в `mu.Lock()` и `mu.Unlock()`.
- 2 Теперь только одна горутина заходит в критическую секцию
- 3 Код стал стабильным — результат всегда 10 000
- 4 **Важно:**
 - `Lock()` и `Unlock()` всегда должны идти в паре
 - Если забыть вызвать `Unlock()` — программа может зависнуть
 - Причина: другие горутины будут ждать освобождения мьютекса

Как избежать ошибок при работе с мьютексами

Хорошая практика — использовать defer сразу после Lock()

```
func increment() {  
    for i := 0; i < 1000; i++ {  
        mu.Lock()  
        counter++  
        mu.Unlock()  
    }  
}
```

Как избежать ошибок при работе с мьютексами

В более сложных функциях мы используем:

```
func increment() {  
    mu.Lock()  
    defer mu.Unlock()  
  
    // операции с общими данными  
    counter++  
}
```


Mutex для защиты словаря

В этом примере мы считаем количество посещений страниц сайта

```
package main
import (
    "fmt"
    "sync"
)
var (
    pageVisits = make(map[string]int)
    mu         sync.Mutex
    wg         sync.WaitGroup
)
func trackVisit(page string) {
    mu.Lock()
    pageVisits[page]++
    mu.Unlock()
    wg.Done()
}
```

```
func main() {
    pages := []string{"home", "about", "home",
        "contact", "home", "about"}

    wg.Add(len(pages))

    for _, page := range pages {
        go trackVisit(page)
    }

    wg.Wait()

    fmt.Println("Visits:", pageVisits)
}
```

Ошибки при работе с мьютексами

Ошибка 1: забыли Unlock()

- Горутины захватывают мьютекс и навсегда остаются в ожидании
- Программа зависает
- Решение — использовать `defer mu.Unlock()` сразу после `mu.Lock()`

Ошибка 2: двойной Unlock()

- Если вызвать `Unlock()` без предварительного `Lock()`
- Программа завершится с паникой:

```
mu.Unlock() // panic: sync: unlock of unlocked mutex
```

Ошибки при работе с мьютексами

Ошибка 3: выход из функции до Unlock()

- Если вызвать return до mu.Unlock(), мьютекс останется заблокирован
- Результат — зависание
- Решение — использовать defer mu.Unlock() сразу после mu.Lock()

Ошибка 4: долгая блокировка

- Не выполняйте тяжёлые операции внутри заблокированного участка
- Это замедлит другие горутины, ожидающие мьютекс

Плохая практика:

```
mu.Lock()  
processBigData()  
mu.Unlock()
```

Ошибки при работе с мьютексами



Ошибка 5: один мьютекс для независимых операций

Если использовать один Mutex для защиты разных, не связанных между собой данных:

- ограничивается параллелизм
- горютины блокируют друг друга без необходимости
- Решение — использовать отдельный мьютекс для каждой независимой структуры или объекта



Mutex и WaitGroup — в чём разница

- WaitGroup используется для ожидания завершения горютин
- Он не защищает доступ к данным
- Mutex — для защиты критической секции с общими ресурсами

WaitGroup управляет завершением

Mutex управляет безопасностью доступа

Что нужно запомнить про sync.Mutex

- Mutex защищает доступ к общим данным
- Используйте `mu.Lock()` и `defer mu.Unlock()`
- Не делайте тяжёлую работу внутри заблокированного участка
- Для независимых объектов — отдельные мьютексы
- Mutex — простой и мощный инструмент, с которого стоит начинать



Ваши вопросы?

Способ решения 3

atomic



7

Способ 3: `sync/atomic`

- Инструмент для атомарных операций
- Предотвращает гонку данных
- Подходит для простых числовых переменных
- Работает быстрее, чем `Mutex`, но сложнее в применении

Зачем нужен `atomic`



Операция `x++` не атомарна:

- читается значение
- изменяется
- записывается обратно



Если две горутины делают это одновременно — возможна потеря данных

- `sync/atomic` обеспечивает:
- атомарность
- неделимость операции
- безопасность при параллельном доступе

Синтаксис и ключевые функции `sync/atomic`



Работает с примитивами:

- `int32`, `int64`
- `uint32`, `uint64`
- `uintptr`
- `unsafe.Pointer`



Часто используемые функции:

- `atomic.AddInt32`, `atomic.AddInt64` — атомарное прибавление
- `atomic.LoadInt32`, `atomic.LoadInt64` — атомарное чтение
- `atomic.StoreInt32`, `atomic.StoreInt64` — атомарная запись
- `atomic.CompareAndSwapInt32`, `atomic.CompareAndSwapInt64` — условная замена значения

Пример: атомарный счётчик

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var counter int64 = 0 // общий счётчик
    var wg sync.WaitGroup // для ожидания завершения
    горутин
```

```
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            // атомарно увеличиваем счётчик на 1
            atomic.AddInt64(&counter, 1)
        }()
    }

    wg.Wait()
    fmt.Println("Итоговое значение счётчика:", counter)
}
```

- Используем `atomic.AddInt64` для безопасного инкремента
- Все 100 горутин работают с переменной без гонки данных

Что происходит в примере

- Объявляем переменную `counter` типа `int64`
 - Запускаем 100 горутин, каждая вызывает `atomic.AddInt64(&counter, 1)`
 - Используется указатель на переменную — обязательно для `atomic`
 - После завершения всех горутин выводим итоговое значение
-
- С `counter++` результат был бы некорректным
 - `atomic.AddInt64` обеспечивает безопасный инкремент без гонки данных



**Зачем нужен atomic, если можно
использовать Mutex?**

Когда использовать `atomic`, а когда — `Mutex`

→ `sync/atomic` — быстрее, чем `Mutex`

- Работает без блокировок
- Использует атомарные инструкции процессора
- Подходит для счётчиков, флагов, переключателей

→ `Mutex` — универсальнее

- Защищает доступ к структурам и сложным данным
- Удобнее в поддержке и масштабировании логики

Compare-And-Swap (CAS)



Функция CompareAndSwap:

- Сравнивает текущее значение переменной с ожидаемым
- Если совпадает — заменяет новым значением
- Возвращает true, если замена выполнена

Пример: Compare-And-Swap (CAS)

Применяется для:

- однократной инициализации
- защиты от повторного выполнения

```
package main

import (
    "fmt"
    "sync/atomic"
)

func main() {
    var flag int32 = 0

    success :=
atomic.CompareAndSwapInt32(&flag, 0, 1)
    if success {
        fmt.Println("Флаг установлен успешно")
    } else {
        fmt.Println("Флаг уже был установлен")
    }
}
```


Типичные ошибки при работе с `atomic`

- Не передали указатель — ошибка компиляции
- Использовали `int` вместо `int32` или `int64` — не работает
- Пытались обновить сразу несколько переменных — атомарность нарушена
- Смешали `atomic` и обычный доступ к переменной — возникает гонка данных

Важно: используйте `atomic` только для одной переменной и только с атомарными операциями

Кейс: подсчёт запросов с atomic

⓪ Задача: считать количество запросов на веб-сервере

```
package main
import (
    "fmt"
    "net/http"
    "sync/atomic"
)
var requestCount int64 = 0
func handler(w http.ResponseWriter, r *http.Request) {
    atomic.AddInt64(&requestCount, 1)
    fmt.Fprintf(w, "Привет! Запрос номер %d\n", atomic.LoadInt64(&requestCount))
}
func main() {
    http.HandleFunc("/", handler)
    fmt.Println("Сервер на порту 8080")
    http.ListenAndServe(":8080", nil)
}
```

Кейс: подсчёт запросов с `atomic`



Решение — `sync/atomic`:

- безопасно
- без блокировок
- подходит для счётчика

Итог по sync/atomic

Подходит для:

- счётчиков
- флагов
- условной замены значений (CAS)

Плюсы:

- высокая скорость
- отсутствие блокировок

Важно соблюдать:

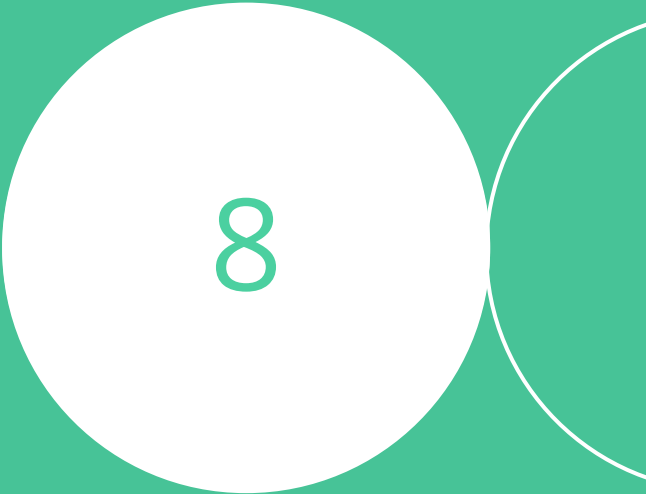
- работу только с указателями
- строгие типы (int32, int64)
- единообразие доступа: либо только atomic, либо Mutex — не смешивать



Ваши вопросы?

Способ решения 4

Каналы



8

Что такое канал

- Канал — механизм передачи данных между горутинами
- Позволяет **синхронизировать выполнение без мьютексов**
- Работает **как конвейер**:
 - одна горутина отправляет данные
 - другая получает их
- Используется для безопасного обмена значениями в конкурентной среде

Принцип работы каналов

- **Отправитель** — горутина, **передающая** данные в канал
- **Получатель** — горутина, **читающая** данные из канала
- **Каналы** блокируют выполнение:
 - если нет получателя — отправитель ждёт
 - если нет данных — получатель ждёт
- **Блокировка обеспечивает синхронизацию между горутинами**

Синтаксис каналов

→ Создание канала:

```
ch := make(chan int)
```

Канал ch передаёт значения типа int

→ Отправка значения в канал:

```
ch <- 5
```

→ Получение значения из канала:

```
value := <-ch
```

Пример: простая работа с каналом

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan int)

    go func() {
        ch <- 42 // отправляем значение в канал
    }()

    val := <-ch // получаем значение из канала
    fmt.Println("Получили значение из канала:", val)
}
```

Пример: простая работа с каналом



Что происходит в примере

- Создаём канал `ch` для передачи `int`
- Запускаем горутину, которая отправляет 42
- Основная горутина читает значение и выводит его



Канал синхронизирует горутин:

- Получатель ждёт, если данных ещё нет
- Отправитель ждёт, если канал занят

Буферизованные и небуферизованные каналы



Небуферизованный канал:

- нет внутреннего хранения
- отправитель ждёт, пока получатель заберёт данные



Буферизованный канал:

- имеет внутренний буфер
- можно отправить несколько значений без ожидания



Пример создания буферизованного канала:

```
ch := make(chan int, 3)
```

Пример: буферизованный канал

- Отправляем 3 значения без блокировки
- Читаем их по очереди
- Канал работает как очередь с фиксированной длиной

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan int, 3)

    ch <- 1
    ch <- 2
    ch <- 3

    fmt.Println(<-ch) // 1
    fmt.Println(<-ch) // 2
    fmt.Println(<-ch) // 3
}
```

Каналы для синхронизации горутин

- Канал может использоваться как способ синхронизации
- Каждая горутина отправляет сигнал в канал по завершении
- Главная горутина получает сигналы и ждёт окончания всех задач

Плюс: не нужно использовать `sync.WaitGroup` — только каналы

Пример: синхронизация с помощью канала

```
package main

import (
    "fmt"
    "time"
)

func worker(done chan bool, id int) {
    fmt.Printf("Горутина %d начала работу\n", id)
    time.Sleep(time.Second)
    fmt.Printf("Горутина %d завершила работу\n", id)
    done <- true
}
```

```
func main() {
    done := make(chan bool)

    for i := 1; i <= 3; i++ {
        go worker(done, i)
    }

    // Ждём, пока все 3 горутины отправят сигнал о
    // завершении
    for i := 1; i <= 3; i++ {
        <-done
    }

    fmt.Println("Все горутины завершены")
}
```

Заккрытие канала

- Канал можно закрыть с помощью `close(ch)`
- Получатели узнают, что новых данных больше не будет
- Особенно важно при чтении в цикле `for range`
- Чтение завершится автоматически при закрытии канала

Пример: чтение до закрытия канала

- Отправляем 5 значений в канал
- Закрываем канал
- Получатель читает данные до конца без блокировки

```
package main

import "fmt"

func main() {
    ch := make(chan int)

    go func() {
        for i := 1; i <= 5; i++ {
            ch <- i
        }
        close(ch) // закрываем канал, сигнализируя о конце передачи
    }()

    for val := range ch {
        fmt.Println(val)
    }
}
```

Потенциальные ошибки и ловушки с каналами

1 **Чтение из закрытого канала** — возвращает нулевое значение

2 **Повторное закрытие канала** — вызывает панику

3 **Отправка в закрытый канал** — вызывает панику

4 **Блокировка на небуферизированном канале без получателя** — горутина «зависает»

5 **Утечки горутин** — если никто не читает из канала, отправитель блокируется навсегда

Оператор `select` в Go

- Позволяет работать с несколькими каналами одновременно
- Ждёт, пока один из каналов станет готов
- Выполняет только один готовый case
- Используется для обработки тайм-аутов, альтернатив и синхронных событий

Пример: select с двумя каналами

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ch1 := make(chan string)
    ch2 := make(chan string)

    go func() {
        time.Sleep(1 * time.Second)
        ch1 <- "Отправлено из ch1"
    }()
```

```
    go func() {
        time.Sleep(2 * time.Second)
        ch2 <- "Отправлено из ch2"
    }()

    for i := 0; i < 2; i++ {
        select {
            case msg1 := <-ch1:
                fmt.Println(msg1)
            case msg2 := <-ch2:
                fmt.Println(msg2)
        }
    }
}
```

Почему каналы — основной способ синхронизации в Go

- Позволяют передавать данные между горутинами напрямую
- Избавляют от явной работы с мьютексами и разделяемой памятью
- Повышают читаемость и надёжность кода
- Снижают риск гонок и блокировок



Принцип Go: **«Не делитесь памятью для общения — общайтесь, чтобы делиться памятью»**

Итоги по каналам

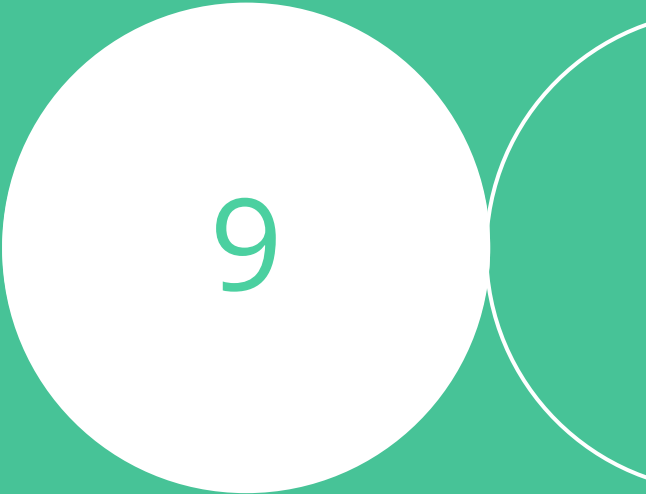
- 1 Безопасный способ передачи данных между горутинами
- 2 Синхронизация достигается через блокирующее поведение
- 3 Каналы бывают буферизованными и небуферизованными
- 4 Заккрытие канала сигнализирует о завершении передачи
- 5 Позволяют строить сложные схемы взаимодействия
- 6 Важно учитывать типичные ошибки:
 - повторное закрытие
 - отправка в закрытый канал
 - блокировки без получателя



Ваши вопросы?

Практический разбор

9

A decorative graphic consisting of two overlapping circles. The circle on the left is white with a thin black outline and contains the number '9' in a black serif font. The circle on the right is light blue with a thin black outline and is partially cut off by the right edge of the slide.

Пример

Параллельная обработка задач с синхронизацией через каналы

Необходимо запустить 5 горутин, каждая из которых возводит число в квадрат, и передаёт результат обратно в главную горутину

- каждая задача выполняется в отдельной горутине
- результаты передаются через канал
- главная горутина собирает и выводит полученные значения
- синхронизация достигается без мьютексов — только через каналы

Пример: параллельная обработка с каналами

```
package main

import (
    "fmt"
)

// Функция, которая принимает число и канал для
// отправки результата
func square(num int, results chan int) {
    squared := num * num
    results <- squared // отправляем результат в канал
}

func main() {
    nums := []int{1, 2, 3, 4, 5}    // исходные данные
```

```
    results := make(chan int, len(nums)) // создаём
    буферизированный канал на 5 значений

    // Запускаем горутин
    for _, n := range nums {
        go square(n, results)
    }

    // Считываем результаты из канала
    for i := 0; i < len(nums); i++ {
        result := <-results
        fmt.Println("Результат:", result)
    }
}
```

Разбор кода: по шагам

- 1 Функция `square` принимает число и канал, в который отправляет квадрат числа
- 2 В `main` задан срез чисел `nums` — данные для обработки
- 3 Канал `results` создаётся с буфером длиной `len(nums)` — это предотвращает блокировки
- 4 В цикле запускаем горутину на каждый элемент: считаем квадрат и передаём в канал
- 5 В главной горутине читаем из канала ровно столько значений, сколько отправили
- 6 Выводим каждый результат на экран

Разбор кода

- Используем буферизированный канал, чтобы избежать блокировок при записи
- Количество чтений равно количеству задач — это предотвращает утечки и зависания
- Код простой, без мьютексов и ручной синхронизации
- Решение безопасное и читаемое — легко масштабируется



Что если убрать буфер у канала?

Разбор кода

→ Что если убрать буфер у канала

```
results := make(chan int)
```

- Отправка в канал будет блокировать горутину до чтения
- Если главная горутина не читает достаточно быстро — возможны задержки
- В текущем примере код не упадёт, но может работать медленнее
- При использовании небуферизированного канала важно строго соблюдать баланс записи и чтения, чтобы избежать взаимных блокировок

Модификация: закрытие канала и range

```
package main
import (
    "fmt"
)
func square(num int, results chan int) {
    results <- num * num
}
func main() {
    nums := []int{1, 2, 3, 4, 5}
    results := make(chan int, len(nums))
    for _, n := range nums {
        go square(n, results)
    }
    close(results) // Нельзя закрывать канал пока горютины работают, здесь пример для демонстрации!

    for res := range results {
        fmt.Println("Результат:", res)
    }
}
```

Ошибка: преждевременное закрытие канала

- Канал можно закрывать только после завершения всех горутин, которые отправляют в него данные
- Если закрыть канал раньше — возможна паника при попытке записи
- Закрытие следует выполнять из главной горутины
- Перед этим нужно убедиться, что все отправляющие горутины завершили работу
- Как правильно закрывать канал
 - Канал следует закрывать только после завершения всех отправляющих горутин
 - Обычно это делает главная горутина, контролирующая выполнение задач
 - Для отслеживания завершения используют `sync.WaitGroup` или аналогичный механизм
 - Сначала — ожидание завершения всех горутин
 - Затем — безопасное закрытие канала

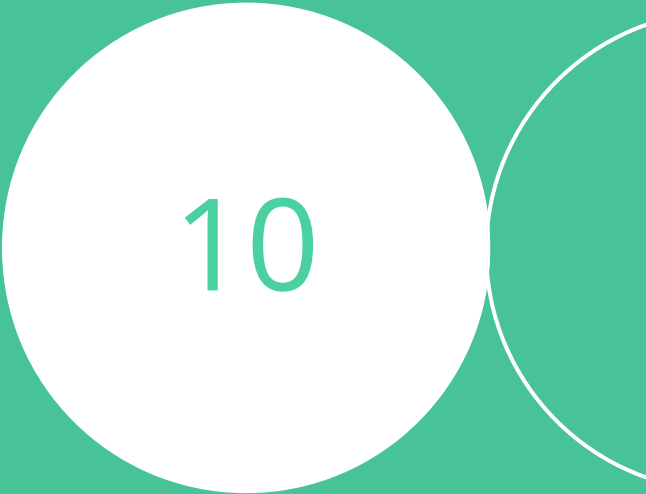
Итог: синхронизация через каналы

- Каналы позволяют удобно собирать данные из нескольких горутин
- Обеспечивают синхронизацию без мьютексов и сложных блокировок
- Важно помнить:
 - Используйте буфер, чтобы избежать блокировок при отправке
 - Читайте ровно столько значений, сколько отправлено
 - Закрывайте канал только после завершения всех отправляющих горутин



Ваши вопросы?

Итоги



10

Что мы изучили

- 1 WaitGroup. Позволяет главной горутине дождаться завершения всех рабочих горутин без сложных блокировок
- 2 Mutex. Защищает общий ресурс от одновременного доступа, предотвращает гонку данных и ошибки синхронизации
- 3 Atomic. Обеспечивает безопасное обновление отдельных переменных без блокировок — быстро, но подходит только для простых случаев
- 4 Каналы. Обеспечивают безопасную передачу данных между горутинами и синхронизацию выполнения без мьютексов

Зачем это нужно знать

- Параллельное программирование повышает производительность и отзывчивость приложений
- Но оно связано с рисками: гонки данных, блокировки, утечки
- Понимание механизмов синхронизации позволяет писать стабильный, безопасный и читаемый код
- Сравнение механизмов
 - WaitGroup — для ожидания завершения горутин, не защищает данные
 - Mutex — защищает общий ресурс, подходит для сложных структур
 - Atomic — быстрые операции с отдельными переменными, без блокировок
 - Каналы — синхронизация через передачу данных, архитектурно выразительный инструмент

Что важно унести с этого занятия

- Синхронизация — не просто техника, а основа корректного многопоточного программирования
- Каждый механизм — это инструмент, важно понимать его назначение и ограничения
- Осваивать синхронизацию лучше всего на практике — пишите простые программы, пробуйте, анализируйте
- Чем больше практики — тем надёжнее и понятнее будет ваш параллельный код

Вопросы для закрепления

- 1 В каких случаях стоит использовать WaitGroup, а когда — Mutex?
- 2 Почему atomic-операции не подходят для всех задач синхронизации?
- 3 Чем каналы отличаются от Mutex по принципу работы и архитектуре?
- 4 Какие ошибки могут возникнуть при неправильной синхронизации в горутинах?



Ваши вопросы?

Синхронизация горутин

Матвей Ефимов

Разработчик учебных программ и эксперт в Go, Python, SQL

